MONITORING · DOMAIN 5 - MONITORING AND ALERTING

Reading the Spark UI and the Query Profiler

Section 5. Where the evidence lives, and how to read it in order.

SECTION 5 of 10 WEIGHT 10% TOPIC Profiling

admin@sthithapragna.com

The profile tells you **what the query did**. The Spark UI tells you **how the work was distributed**. Slow queries usually need both.

What the query summary gives you before you dig

METRIC	WHAT IT TELLS YOU
Wall-clock duration	Total elapsed time, broken into scheduling, optimisation and pruning, and execution.
Aggregated task time	Combined time across all cores of all nodes.
Query metrics	Includes filter icons showing the percent of data pruned during scanning.
Top operators	The most expensive operators in the query, ranked.
Input and output	How much data was actually read and written.

ON THE EXAM

The breakdown of wall-clock time matters. A query that spent most of its time in scheduling has a queueing problem, not a query problem.

Aggregated task time against wall-clock

- **Aggregated task time much longer than wall-clock** means work ran in parallel. That is healthy.
- **Aggregated task time shorter than wall-clock** means tasks were **waiting for available nodes**.
- So the second case points at compute contention rather than at the query.
- Both directions are documented behaviour. Neither is an error.

> ON THE EXAM
>
> Comparing these two numbers is the fastest way to tell a slow query from a query that was waiting its turn.

When there is no profile at all

- **A query served from the query cache has no profile.** You get a message saying it is not available.
- The documented workaround: make a **trivial change** to the query, such as altering or removing the LIMIT.
- That bypasses the cache and forces a real execution you can profile.
- It is also a reminder that a fast second run may be telling you nothing about the query.

> ON THE EXAM
>
> Profiling a cached query is measuring the cache. Change something trivial, then profile the run that actually did work.

Six operations, and which one to worry about

OPERATOR	WHAT IT DID
Scan	Read data from a source and produced rows.
Filter	Applied criteria and returned a subset.
Join	Combined rows from several relations.
Union	Concatenated rows from relations sharing a schema.
Hash / Sort	Grouped rows by a key and applied an aggregate.
Shuffle	Redistributed data between executors. Explicitly called expensive.

ON THE EXAM

The documentation singles out shuffle as expensive because it moves data between executors. That is module 13's point, stated here as a reading instruction.

Three metrics, one at a time

- The graph can display **time spent**, **memory peak**, or **rows**. Switch between them.
- Time spent finds the slow operator. **Rows** finds the operator producing more than it should.
- An operator emitting far more rows than it consumed is the signature of an **exploding join**.
- The Top operators panel gives you the same ranking without reading the graph.

> ON THE EXAM
>
> Switching the graph to rows is the underused move. Time tells you where the cost landed; rows often tells you why.

Verbose mode, and a Photon caveat

- Metrics for some operations are **hidden by default**, on the basis that they are unlikely to be bottlenecks.
- **Enable verbose mode** from the kebab menu to see every operation and additional metrics.
- **Non-Photon operations executed as a group share metrics**, so each shows the parent operator's value.
- So identical numbers across several operators may be an artefact, not a coincidence.

> ON THE EXAM
>
> If several adjacent operators report exactly the same figure, check whether they are grouped before drawing a conclusion from it.

Performance insights

- Databricks surfaces insights automatically, **ranked by projected effect on total task duration**.
- They appear as a summary in the query details panel, and in full in a **Performance insights** tab.
- Where actionable, an **Optimize** button opens Genie Code, which rewrites the query for your approval.
- For table or compute changes it summarises the recommendation in plain language instead.

> ON THE EXAM
>
> Ranked by projected effect is the useful part. It tells you which of five suggestions is worth doing first.

Two routes, and one that does not persist

- If the other person has **CAN MANAGE** on the query, share the URL directly.
- Otherwise **download the profile as JSON**, and they import it from the query history.
- An imported profile is loaded into the **browser session only**. It does not persist in the workspace.
- That makes JSON the route for anyone outside the workspace, including support.

> ON THE EXAM
>
> The JSON route works for someone who is not a workspace member at all, which is how a profile reaches Databricks support.

The profile is not only in query history

- From the **SQL editor**, via the link showing elapsed time and rows returned.
- From a **notebook** attached to a warehouse or serverless compute, via See performance.
- From the **Lakeflow pipelines UI**, in its Query History tab.
- From the **jobs UI**, for jobs run on warehouses and serverless compute.

> ON THE EXAM
>
> That pipelines entry point matters. It means a declarative pipeline's queries can be profiled like any other, which is not obvious.

Open in Spark UI

- For Databricks SQL queries, the kebab menu offers **Open in Spark UI**.
- That is the move once the profile has told you **which operator** is expensive.
- The profile works in operators. The Spark UI works in **stages and tasks**.
- Skew and spill are task-level properties, so the profile alone will not show them clearly.

> ON THE EXAM
>
> Use the profile to find the operator, then the Spark UI to find out what the tasks inside it were doing.

Start at the longest stage, and get three numbers

- Open **Stages**, sort by duration descending, and take the longest one. Everything else is noise until you understand it.
- Inside it, read the **median task duration**, the **maximum**, and the **shuffle read** figures.
- The **ratio of maximum to median** is your first discriminator.
- Everything else in that interface is confirmation, not discovery.

> ON THE EXAM
>
> Discipline matters more than knowledge here. The Spark UI shows hundreds of numbers, and three of them do most of the work.

What the pattern of task times means

PATTERN	LIKELY CAUSE
A few tasks far slower than the median, with skewed shuffle read	Data skew
Spill appearing on most tasks, not just outliers	Memory pressure
All tasks uniformly slow	Insufficient resources, or too few partitions
High garbage collection time across executors	Memory pressure again

ON THE EXAM

A few slow tasks means skew. Uniformly slow tasks means resources. That single distinction resolves most of these questions.

Wall-clock is not evidence

- A single faster run is not validation. Cluster warmth, caching and cluster load all move wall-clock time.
- Confirm that the **underlying metric moved**: less spill, lower garbage collection, a balanced max-to-median ratio.
- Before manual tuning, check whether **adaptive execution** already handled it. Much tuning advice predates it being default.
- And remember that a cached query has no profile at all, so a fast rerun may not have run.

> ON THE EXAM
>
> This is the same instinct as module 13. Prove the mechanism changed, not just that the number got smaller.

What to carry into the exam

- You need **CAN MONITOR** on the warehouse, or ownership of the query, to see a profile.
- **Cached queries have no profile.** Change the LIMIT to force a real run.
- **Aggregated task time shorter than wall-clock** means tasks waited for nodes.
- Switch the graph to **rows** to find an exploding join. Insights are ranked by projected effect.
- Profile finds the operator. **Spark UI** finds the task pattern: few slow tasks is skew, all slow is resources.

> ON THE EXAM
>
> Two tools, two altitudes. Reach for the profile first, and cross to the Spark UI once you know which operator to interrogate.

> An analyst reports that a dashboard query takes 40 seconds. An engineer opens the query profile and sees the wall-clock duration is 38 seconds, while the aggregated task time is 6 seconds.
>
> The query itself is a simple aggregate over a well-clustered table, and the operators show no obvious problem.

What should they investigate?

A The query logic, since a simple aggregate should not take 38 seconds.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

C Data skew, since one task must be dominating the total.

D File layout, since poor data skipping would explain the elapsed time.

◆ PAUSE AND DECIDE

What should they investigate?

A The query logic, since a simple aggregate should not take 38 seconds.

The work took six seconds. The query is not the problem.

B Compute availability, since aggregated task time far below wall-clock means tasks waited for nodes.

Documented behaviour: task time shorter than wall-clock indicates tasks waiting for available nodes.

C Data skew, since one task must be dominating the total.

Skew would show as a long aggregated task time with an uneven distribution, not a short one.

D File layout, since poor data skipping would explain the elapsed time.

Bad pruning means more data scanned, which raises task time rather than lowering it.

ON THE EXAM

Six seconds of actual work inside thirty-eight seconds of elapsed time. Thirty-two seconds were spent somewhere other than computing.

> An engineer changes a query, reruns it, and sees it complete in two seconds instead of ninety. Pleased, they open the query profile to document the improvement.
>
> The profile page says a query profile is not available.

What has happened?

A The optimisation was so effective that no profile was generated.

B The result came from the query cache, which produces no profile. The two-second run may not prove anything.

C Profiles are only generated for queries exceeding a minimum duration.

D The engineer lacks CAN MONITOR permission on the warehouse.

◆ PAUSE AND DECIDE

What has happened?

A The optimisation was so effective that no profile was generated.

Profiles are produced regardless of duration.

B The result came from the query cache, which produces no profile. The two-second run may not prove anything.

Documented: no profile exists for a cached result. Change the LIMIT trivially to force a real execution.

C Profiles are only generated for queries exceeding a minimum duration.

No such threshold exists.

D The engineer lacks CAN MONITOR permission on the warehouse.

That gives a permission error, not a 'not available' message.

ON THE EXAM

The missing profile is itself the diagnosis. It tells you the query did not actually run.

> A join between two large tables is the slowest operator in a profile. The engineer opens the Spark UI, sorts stages by duration, and examines the longest stage.
>
> Median task duration is 4 seconds. Maximum task duration is 26 minutes. Shuffle read is similarly lopsided. Spill appears on only two tasks.

What is the diagnosis?

A Memory pressure, because spill is present.

B Data skew, because a few tasks far exceed the median and shuffle read is also skewed.

C Insufficient cluster resources, because the maximum duration is very high.

D Too few shuffle partitions, because the median is only 4 seconds.

◆ PAUSE AND DECIDE

What is the diagnosis?

A. Memory pressure, because spill is present.
Spill on two tasks out of many is a consequence of skew. Memory pressure shows spill on most tasks.

B. Data skew, because a few tasks far exceed the median and shuffle read is also skewed.
Both conditions hold: an enormous max-to-median ratio, and shuffle read lopsided the same way.

C. Insufficient cluster resources, because the maximum duration is very high.
Resource limits make all tasks slow. The median here is four seconds.

D. Too few shuffle partitions, because the median is only 4 seconds.
Too few partitions would make tasks uniformly large, raising the median.

ON THE EXAM

Spill on two tasks is the trap. Skew causes localised spill; memory pressure causes widespread spill.

> A query takes eleven minutes. The profile shows a join operator consuming most of the time. The engineer switches the graph metric from time spent to rows and sees the join consuming 4 million rows and emitting 900 million.
>
> The join condition matches on a customer identifier.

What does this indicate?

A. An exploding join, likely caused by duplicate keys on one side.

B. A broadcast join that failed and fell back to sort merge.

C. Poor data skipping, since too many rows were scanned.

D. Data skew, since the row count is unbalanced.

◆ PAUSE AND DECIDE

What does this indicate?

A An exploding join, likely caused by duplicate keys on one side.

Output vastly exceeding input is the definition. Duplicate keys multiply matching rows, and the docs name exploding joins explicitly.

B A broadcast join that failed and fell back to sort merge.

A strategy change alters cost, not row counts.

C Poor data skipping, since too many rows were scanned.

That would show at the Scan operator, not as growth across a join.

D Data skew, since the row count is unbalanced.

Skew is about distribution across tasks, not about total output exceeding input.

ON THE EXAM

Switching the graph to rows is what surfaced this. Time alone would have told you the join was slow and not why.

> A team is debugging a failing query with Databricks support. Support asks for the query profile. The support engineer is not a member of the workspace.
>
> A team member tries to send the profile URL and support cannot open it.

What is the correct approach?

A Grant the support engineer CAN MONITOR on the warehouse so the URL works.

B Download the query profile as JSON and send that; support imports it into their own session.

C Take screenshots of each operator in the DAG.

D Export the query history system table and send the relevant rows.

◆ PAUSE AND DECIDE

What is the correct approach?

A Grant the support engineer CAN MONITOR on the warehouse so the URL works.

Adds an external identity to the workspace to share one diagnostic, when a file does the job.

B Download the query profile as JSON and send that; support imports it into their own session.

The documented route for anyone without CAN MANAGE or outside the workspace entirely.

C Take screenshots of each operator in the DAG.

Loses every metric that is not on screen, and cannot be explored.

D Export the query history system table and send the relevant rows.

Query history is not the profile, and module 17 warns against exporting system table data.

ON THE EXAM

The JSON route exists precisely for people outside the workspace. Note it loads into their browser session and does not persist.

> An engineer is asked why a pipeline's gold materialized view refresh is slow. They say query profiles are only available for Databricks SQL queries, so the pipeline cannot be profiled and they will have to read cluster logs instead.
>
> The pipeline runs on serverless.

Is that correct?

A Yes. Query profiles apply only to SQL warehouse queries.

B No. The Lakeflow pipelines UI has a Query History tab from which profiles can be opened.

C No, but only if the pipeline is switched to a SQL warehouse first.

D Yes, unless verbose mode is enabled on the pipeline.

◆ PAUSE AND DECIDE

Is that correct?

A Yes. Query profiles apply only to SQL warehouse queries.

Profiles are reachable from the pipelines UI, the jobs UI and notebooks as well.

B No. The Lakeflow pipelines UI has a Query History tab from which profiles can be opened.

A documented entry point. Pipeline queries are profiled like any others.

C No, but only if the pipeline is switched to a SQL warehouse first.

No such change is needed.

D Yes, unless verbose mode is enabled on the pipeline.

Verbose mode reveals hidden metrics within a profile. It does not create one.

ON THE EXAM

A common and costly assumption. Reaching for cluster logs when a profile is one tab away wastes a great deal of time.